

Motorcycle Traffic Violation Detection System

Automated detection of helmet non-compliance and motorcycle overloading using YOLOv8

Model	Overall mAP@0.5	Dataset	Training Platform
YOLOv8n	0.926	578 images	Google Colab · Tesla T4

Submit To
Omar Farooq

Phase 1 — Problem Definition

CUSTOM DOMAIN — OPTION E

What objects does the system detect?

The system detects four classes from road footage images and video:

helmet

motorcycle_normal

motorcycle_overloaded

no_helmet

Why is this problem useful?

Motorcycle traffic violations — particularly helmet non-compliance and triple riding (3+ riders on one motorcycle) — are among the most common causes of road accident fatalities in Pakistan. These violations are currently monitored manually by traffic police at checkpoints, which is slow, inconsistent, and limited in coverage.

Punjab's Safe City Authority (PSCA) and PITB are actively developing AI-based e-challan features to automate traffic enforcement. Helmet detection and overloading detection are explicitly listed on their planned deployment roadmap. This project builds a working prototype directly aligned with those government priorities.

Where can this system be applied?

- Roadside fixed cameras at intersections and traffic checkpoints
- Punjab Safe City camera network (PSCA infrastructure)
- Traffic police monitoring stations
- Motorway and highway surveillance systems
- School and hospital zone safety monitoring

Target classes

CLASS ID	CLASS NAME	DESCRIPTION	VIOLATION?
0	helmet	Safety helmet worn by a motorcycle rider	No — compliant
1	motorcycle_normal	Motorcycle carrying 1–2 riders	No — compliant
2	motorcycle_overloaded	Motorcycle carrying 3 or more riders	Yes — violation
3	no_helmet	Rider's bare head without a helmet	Yes — violation

Final output

The system produces annotated images and videos with color-coded bounding boxes, class labels, and confidence scores overlaid on each detected object. Violation classes (no_helmet, motorcycle_overloaded) are highlighted in distinct colors for immediate visibility. For video input, the system uses ByteTrack object tracking to assign persistent IDs to each motorcycle across frames.

Phase 2 — Data Collection

578 IMAGES · 4 CLASSES · MANUALLY ANNOTATED

Images were collected from multiple publicly available datasets on Roboflow Universe and Kaggle. No pre-existing annotations were used — all 578 images were manually reviewed, selected, and re-

annotated from scratch to ensure annotation quality and class consistency.

SOURCE	PLATFORM	USAGE
Helmet and no-helmet rider detection datasets	Roboflow Universe	helmet, no_helmet class images
HelmetViolations dataset (pkdarabi)	Kaggle	Supplementary helmet/no_helmet
Triple ride detection datasets	Roboflow Universe	motorcycle_overloaded images
General motorcycle detection datasets	Roboflow Universe / Kaggle	motorcycle_normal images

All images were manually reviewed before annotation. Low-quality images (extreme blur, very dark, objects below 15×15 pixels, objects more than 50% cut off at frame edges) were excluded. The final 578 images represent clean, annotatable examples across all four classes.

Class distribution

CLASS	ANNOTATED INSTANCES	TRAINING IMAGES
helmet	442	405 train 116 val 57 test
motorcycle_normal	594	
motorcycle_overloaded	181	
no_helmet	978	

Phase 3 — Data Annotation

ROBOFLOW ANNOTATE · YOLO FORMAT · MANUALLY REVIEWED

All 578 images were annotated manually using Roboflow Annotate. Annotations were exported in YOLOv8 format (normalized coordinates). Each image was reviewed after annotation to correct any loose boxes, missed objects, or incorrect class labels.

Annotation rules followed

- **helmet:** tight box around the helmet shell only — includes visor and chin strap, excludes neck and shoulders
- **no_helmet:** box around the rider's head from top of head to chin — only annotated when confirmed to be a motorcycle rider
- **motorcycle_normal:** full motorcycle box from front wheel to rear wheel when 1–2 riders are visible
- **motorcycle_overloaded:** same box dimensions as motorcycle_normal but labeled when 3 or more riders are clearly visible
- Objects more than 50% cut off at the frame edge were not annotated
- Ambiguous cases (unclear rider count, unidentifiable head) were skipped rather than guessed

YOLO label format

Each image has a corresponding .txt file with one line per annotated object:

```
<class_id> <x_center> <y_center> <width> <height>  
Example: 0 0.512 0.460 0.210 0.180
```

All coordinate values are normalized between 0 and 1 relative to image dimensions.

Phase 4 — Dataset Organization

YOLO FORMAT · 70/20/10 SPLIT

The dataset was exported from Roboflow with a 70/20/10 train/validation/test split and organized into the required folder structure:

```
cv_project/  
├─ dataset/  
│   ├── images/  
│   │   ├── train/ (405 images)  
│   │   ├── val/   (116 images)  
│   │   └─ test/   (57 images)  
│   └─ labels/  
│       ├── train/  
│       ├── val/  
│       └─ test/  
└─ data.yaml
```

```
|─ notebooks/training_notebook.ipynb
|─ outputs/training_results|predictions|demo_results
|─ models/best.pt
|─ report/final_report.pdf
|─ error_analysis/error_analysis.md
|─ README.md
|─ requirements.txt
|─ annotation_guidelines.txt
|─ data_sources.txt
```

Phase 5 — Dataset Configuration

DATA.YAML

```
path: /content/drive/MyDrive/cv_project/dataset
train: images/train
val: images/val
test: images/test
nc: 4
names: ['helmet', 'motorcycle_normal', 'motorcycle_overloaded', 'no_helmet']
```

FIELD	MEANING
path	Root directory of the dataset on Google Drive
train	Path to training images relative to root
val	Path to validation images relative to root
test	Path to test images relative to root
nc	Number of classes (4)
names	Class names in order — class ID 0 = helmet, 1 = motorcycle_normal, etc.

Phase 6 — Google Colab Setup

All training was performed on Google Colab using a Tesla T4 GPU (14,913 MiB VRAM). Google Drive was mounted to persist all outputs across sessions.

```
# Mount Drive
from google.colab import drive
drive.mount('/content/drive')

# Install Ultralytics
!pip install -U ultralytics

# GPU check
import torch
print("GPU Available:", torch.cuda.is_available())
!nvidia-smi
```

GPU confirmed: Tesla T4 · 14,913 MiB · CUDA available · PyTorch 2.11.0+cu128

Phase 7 — Model Training

6 EXPERIMENTS · YOLOV8N / YOLOV8S

Six training experiments were conducted to identify the optimal model configuration. All experiments used pretrained ImageNet weights as the starting point.

#	MODEL	EPOCHS	BATCH	CLS	MAP@0.5	NOTES
1	YOLOv8n	30	8	default	0.919	Baseline
2	YOLOv8n ★	40	16	default	0.926	Best model
3	YOLOv8n	40	8	default	0.923	
4	YOLOv8n	75	8	1.5	0.916	Overfitting
5	YOLOv8n	75	16	default	0.916	Overfitting
6	YOLOv8s	75	8	1.5	0.906	Worst — overfitting

Key finding: YOLOv8s (11M parameters) performed worse than YOLOv8n (3M parameters) on this dataset. With 578 training images, the larger model overfits while the smaller model generalizes better. Running more than 40 epochs also caused mild overfitting. The optimal configuration was YOLOv8n at 40 epochs with batch size 16.

Winning model training command

```
model = YOLO("yolov8n.pt")
results = model.train(
    data="/content/drive/MyDrive/cv_project/dataset/data.yaml",
    epochs=40,
    imgsz=640,
    batch=16,
    pretrained=True,
    project=TRAINING_DIR,
    name="YOLOv8n_40epochs_16Batch"
)
```

Training time: 0.121 hours ($\approx$ 7.3 minutes) on Tesla T4 GPU

Training curves — best model

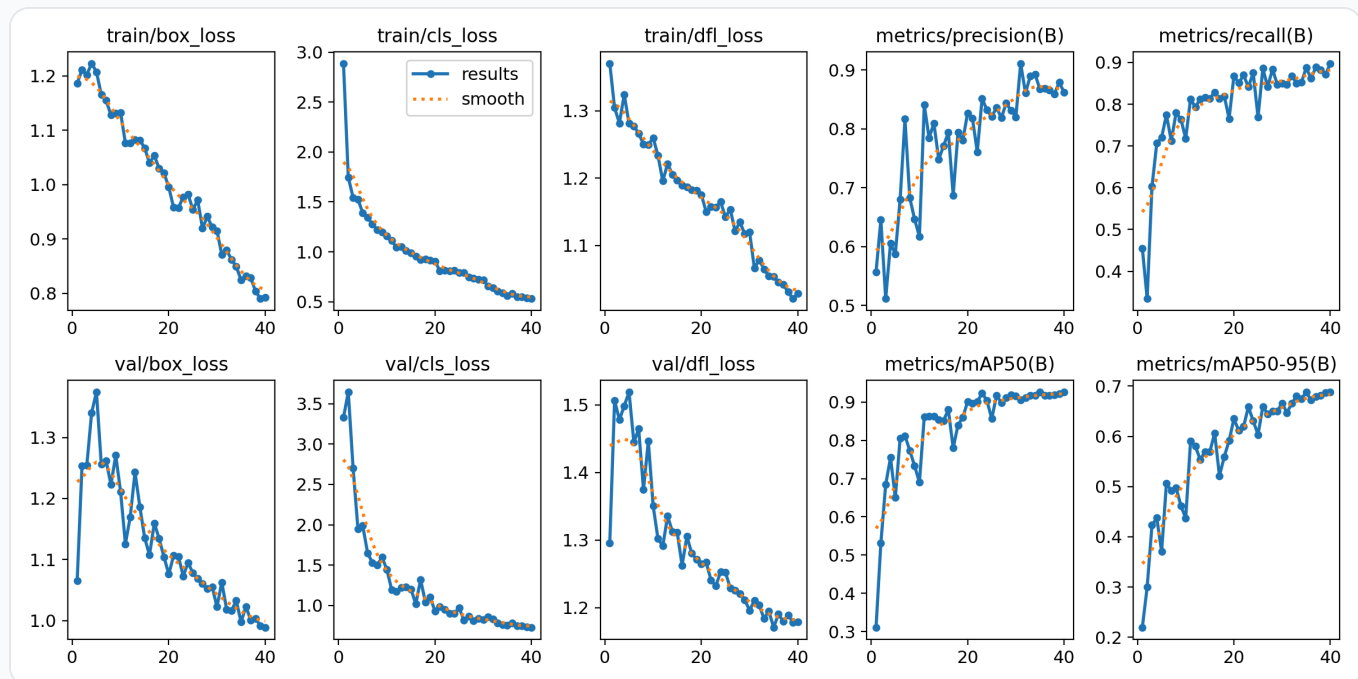


Figure 1 — Training and validation loss curves across 40 epochs. All losses decrease smoothly. mAP50 and mAP50-95 both converge by epoch 35-40, confirming 40 epochs is the correct stopping point for this dataset size.

Phase 8 — Model Evaluation

PRECISION · RECALL · MAP@0.5 · MAP@0.5:0.95

Overall metrics

0.869

PRECISION

0.890

RECALL

0.926

MAP@0.5

0.688

MAP@0.5:0.95

What each metric means

PRECISION — 0.869

Of all the detections the model made, 86.9% were correct. A precision of 0.869 means that when the model says "this is a no_helmet violation", it is right 86.9% of the time. High precision reduces false accusations — compliant riders are not incorrectly flagged.

RECALL — 0.890

Of all the actual violations present in the images, the model found 89.0% of them. A recall of 0.890 means 89 out of every 100 real violations are detected. For a safety enforcement system, high recall is critical — missing violations is worse than a false alarm.

MAP@0.5 — 0.926

Mean Average Precision at 50% IoU threshold. This is the primary evaluation metric for object detection. A box counts as correct if it overlaps the ground truth by at least 50%. 0.926 is considered excellent performance and indicates the model reliably detects and classifies objects.

MAP@0.5:0.95 — 0.688

Average mAP across IoU thresholds from 0.5 to 0.95. This measures bounding box precision more strictly. The gap between 0.926 and 0.688 is normal for road footage — objects appear at varied distances and angles, making perfectly tight boxes harder to achieve consistently.

Per-class performance

CLASS	PRECISION	RECALL	MAP@0.5	MAP@0.5:0.95
helmet	0.837	0.886	0.900	0.644
motorcycle_normal ★ best	0.953	0.918	0.968	0.791
motorcycle_overloaded	0.862	0.895	0.928	0.775
no_helmet (weakest)	0.823	0.860	0.910	0.541

Best class: motorcycle_normal at 0.968 mAP@0.5 — expected, as this was the most represented class in training (594 instances) and motorcycles have distinctive visual features.

Weakest class: no_helmet at 0.910 mAP@0.5 and 0.541 mAP@0.5:0.95. The mAP@0.5:0.95 gap reveals that while the model detects no_helmet cases, the bounding boxes are less precise than other classes. Bare heads are more visually varied than physical helmet objects — different hair types, skin tones, and angles all contribute to this lower precision.

Confusion matrix

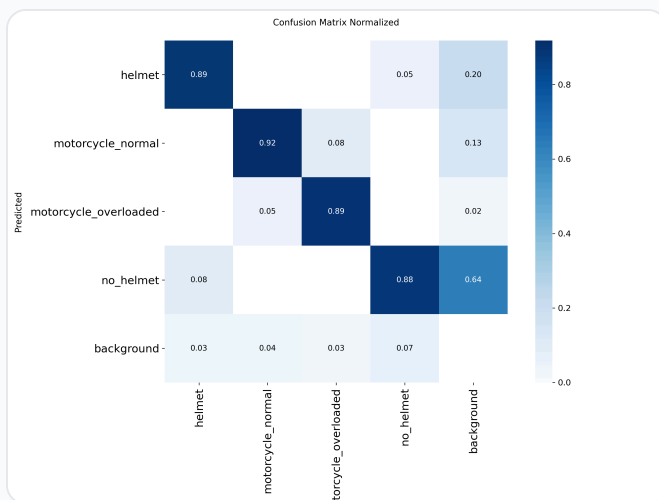


Figure 2 — Normalized confusion matrix. Diagonal values show correct predictions. The no_helmet row shows 0.64 predicted as background — the most significant error pattern.

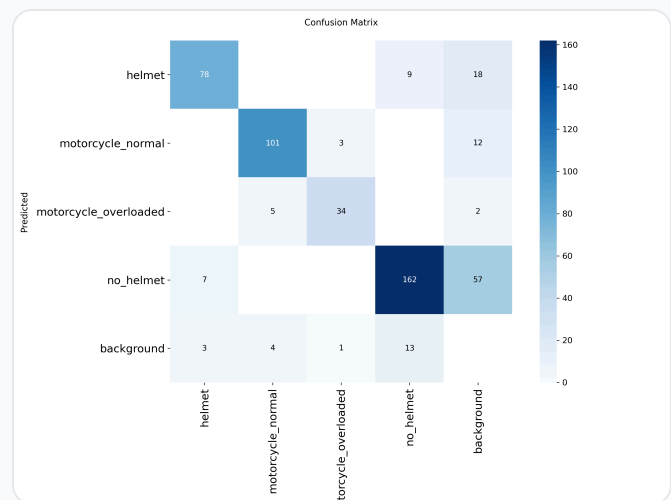


Figure 3 — Raw confusion matrix showing actual counts. no_helmet: 162 correct, 57 missed as background.

Precision-Recall curve

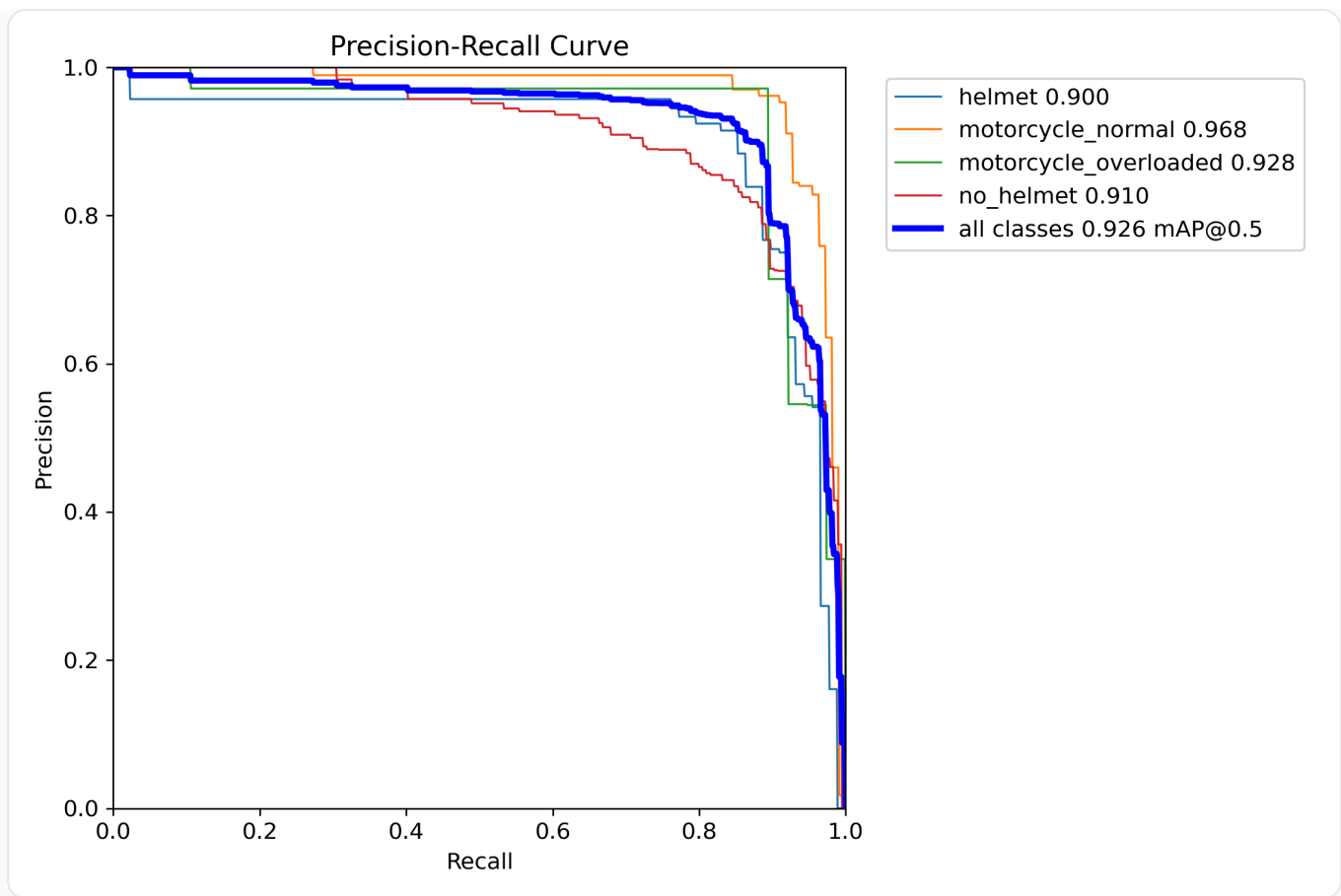


Figure 4 — Precision-Recall curve. All classes maintain high precision across most of the recall range. *motorcycle_normal* (orange) achieves the highest area under the curve at 0.968. The sharp drop at high recall values is typical for small-object detection tasks.

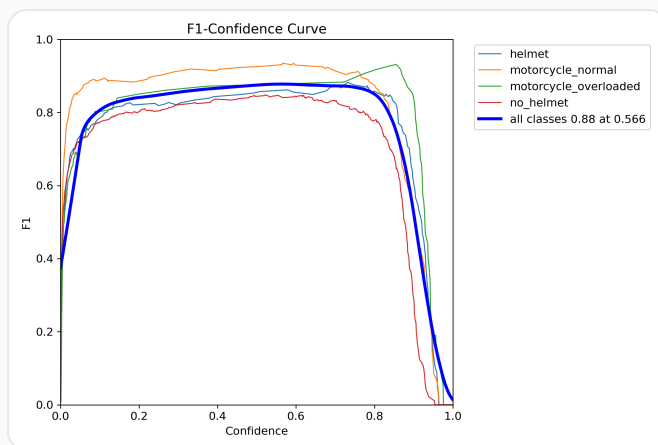


Figure 5 — F1-Confidence curve. Best overall F1 of 0.88 achieved at confidence threshold 0.566. This is the recommended operating threshold for balanced precision and recall.

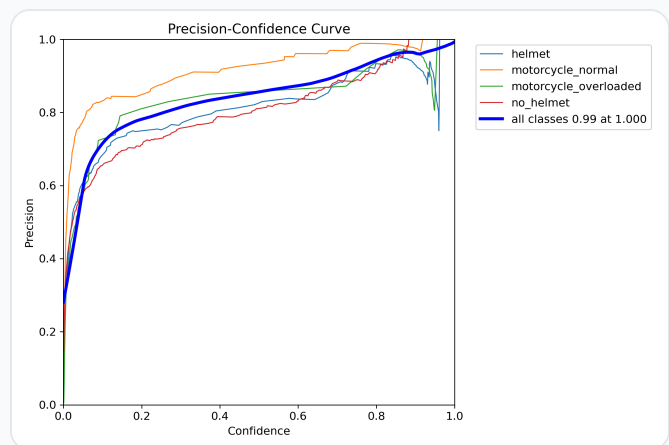


Figure 6 — Precision-Confidence curve. Precision reaches 0.99 at high confidence thresholds — the model is very accurate when it is confident.

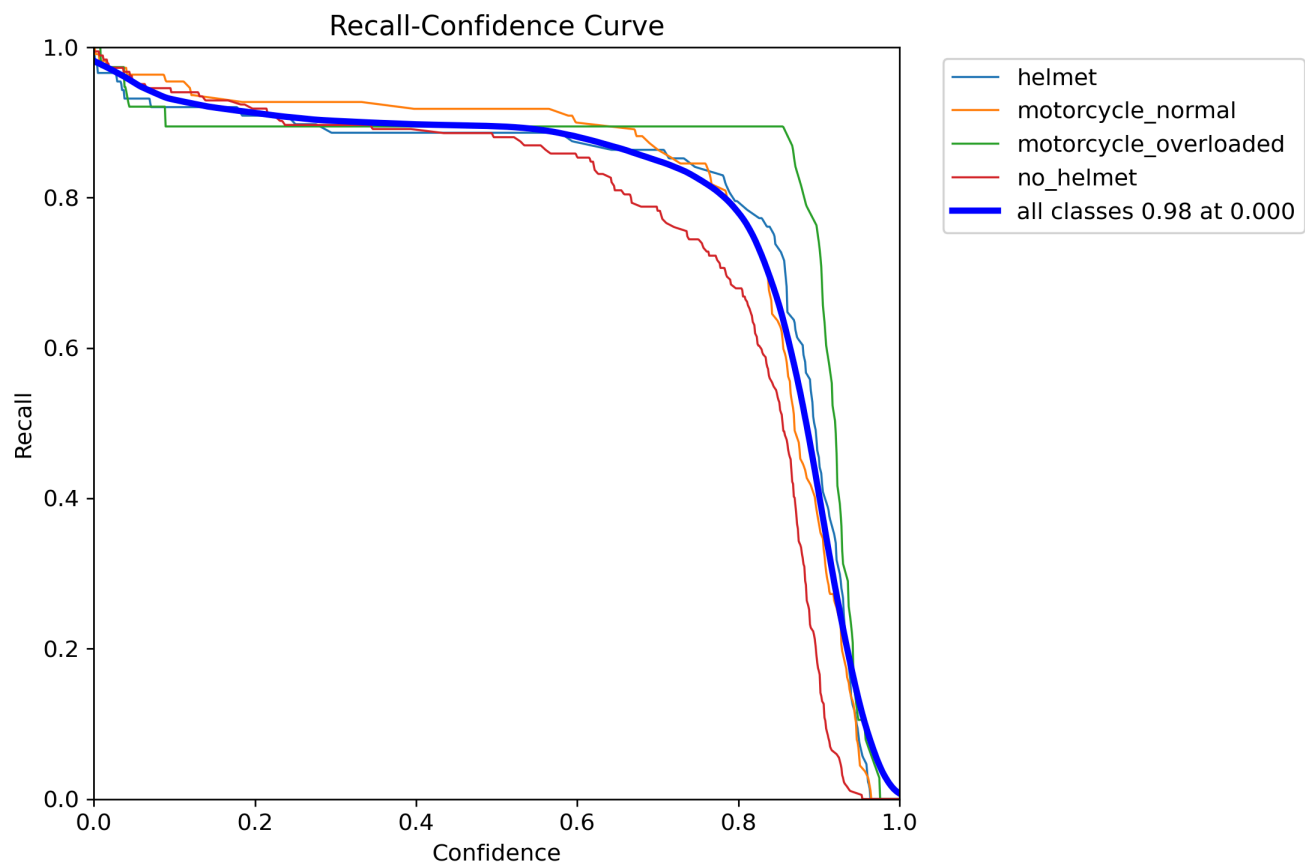


Figure 7 — Recall-Confidence curve. At low confidence thresholds recall reaches 0.98 — nearly all objects are detected. As the threshold increases, recall drops, showing the precision-recall tradeoff.

Phase 9 — Inference on New Data

57 TEST IMAGES · CONF=0.25 · AGNOSTIC NMS

The trained model was evaluated on 57 unseen test images — images the model had never seen during training or validation. Inference was run with a confidence threshold of 0.25 and agnostic NMS (iou=0.4) to prevent duplicate detections of the same object across different classes.

```
model = YOLO("models/best.pt")
results = model.predict(
    source="dataset/images/test",
    conf=0.25,
    iou=0.4,
    agnostic_nms=True,
    save=False
)
```

A custom OpenCV visualization was applied to all predictions: class-specific colors (orange = helmet, green = motorcycle_normal, red = motorcycle_overloaded, magenta = no_helmet), colored label backgrounds for readability, and confidence scores displayed on every box.

Inference speed: 3.0ms per image on Tesla T4 GPU — approximately 333 frames per second theoretical throughput, well within real-time processing requirements for road camera systems.

Phase 10 — Error Analysis

13 ERRORS MANUALLY REVIEWED

10	2	1
Wrong class (helmet ↔ no_helmet)	False positives	False negative (missed)

#	IMAGE	ACTUAL OBJECT	MODEL PREDICTION	ERROR TYPE	POSSIBLE REASON
1	image_1.jpg	helmet	no_helmet	Wrong class	Helmet style made it visually ambiguous at this angle
2	image_2.jpg	helmet	no_helmet	Wrong class	Similar appearance between helmet surface and human hair
3	image_3.jpg	background object	no_helmet	False positive	Random object shape resembled a human head
4	image_4.jpg	motorcycle_overloaded	motorcycle_normal	Wrong class	Overlapping rider bodies made motorcycle

#	IMAGE	ACTUAL OBJECT	MODEL PREDICTION	ERROR TYPE	POSSIBLE REASON
					appear less crowded
5	image_5.jpg	no_helmet	not detected	False negative	Rider was small and partially occluded — confidence below threshold
6	image_6.jpg	person (background)	motorcycle_normal	False positive	Standing person misidentified in motorcycle scene context
7	image_7.jpg	no_helmet	helmet	Wrong class	Rider's hair around head area resembled helmet shape
8	image_8.jpg	no_helmet	helmet	Wrong class	Low-profile helmet-like hairstyle at road camera distance
9	image_9.jpg	helmet	no_helmet	Wrong class	Background clutter — helmet blended with surroundings
10	image_10.jpg	helmet	no_helmet	Wrong class	Poor lighting reduced visual distinction between classes
11	image_11.jpg	no_helmet	helmet	Wrong class	Head shape and scene

#	IMAGE	ACTUAL OBJECT	MODEL PREDICTION	ERROR TYPE	POSSIBLE REASON
					context misled the model
12	image_12.jpg	no_helmet	helmet	Wrong class	no_helmet instance blended visually with helmet class
13	image_13.jpg	no_helmet	helmet	Wrong class	Similar appearance between head covering and helmet shell

Key findings

FINDING 1 — HELMET VS NO_HELMET CONFUSION IS THE DOMINANT FAILURE MODE (76.9%)

10 of 13 errors involve confusing helmet and no_helmet. This is the most critical failure for a safety enforcement system. The root cause is insufficient diversity in training data — the model needs more examples covering different helmet styles, hair types, and lighting conditions.

FINDING 2 — BACKGROUND CONFUSION ON NO_HELMET (CONFUSION MATRIX: 0.64 MISSED AS BACKGROUND)

The normalized confusion matrix shows 64% of background instances being predicted as no_helmet and 20% of helmets missed as background. This is a data coverage problem — the model has not seen enough variation in how objects appear against different backgrounds.

FINDING 3 — MOTORCYCLE_OVERLOADED CONFUSION WITH MOTORCYCLE_NORMAL (5% ERROR RATE)

When multiple riders overlap or are partially occluded, the model struggles to correctly classify the load. This is an inherent limitation of single-frame detection without explicit person counting — the model learns visual density patterns rather than actually counting riders.

Suggested improvements

- **Collect 300+ more diverse helmet/no_helmet images** — different helmet styles, hair types, distances, and lighting conditions to reduce the 76.9% wrong-class error rate
- **Apply targeted augmentation** — brightness reduction, increased blur, and hue shifts specifically targeting the lighting and distance failure scenarios
- **Increase motorcycle_overloaded training data** — only 181 examples were available; 150+ additional side-on shots showing 3 riders clearly would reduce normal/overloaded confusion
- **Train with imgs=1280** — higher input resolution would improve detection of small and distant objects, addressing the false negative problem
- **Deploy with lower confidence threshold for no_helmet** — $\text{conf}=0.20$ specifically for violation classes to reduce false negatives at the cost of acceptable false positives

Phase 11 — Demo

VIDEO INFERENCE · BYTETRACK

Video inference was performed using YOLOv8's built-in ByteTrack integration. Each motorcycle in the video receives a persistent tracking ID across frames, allowing the system to monitor individual vehicles continuously rather than frame-by-frame.

```
model = YOLO("models/best.pt")
results = model.track(
    source="video.mp4",
    tracker="bytetrack.yaml",
    conf=0.25,
    save=True
)
```

The annotated video output shows bounding boxes, class labels, confidence scores, and persistent vehicle IDs across all frames. Saved to `outputs/demo_results/`.

Conclusion

This project successfully built and evaluated a motorcycle traffic violation detection system using YOLOv8, achieving an overall mAP@0.5 of 0.926 across four classes. Six training experiments were conducted, revealing that model size must match dataset size — the smaller YOLOv8n generalized better than YOLOv8s on 578 training images.

The system directly addresses two violation types on Punjab Safe City's e-challan deployment roadmap: helmet non-compliance and motorcycle overloading. The main limitation identified is the helmet vs no_helmet confusion, which is a data diversity problem solvable through additional training data collection.

Tools used: Python · Google Colab · Tesla T4 GPU · YOLOv8 (Ultralytics 8.4.153) · PyTorch 2.11.0 · Roboflow · OpenCV · Matplotlib

GitHub: <https://github.com/hachi-man/motorcycle-violation-detection>

Submitted to: Omar Farooq — Associate AI/ML Engineer, PITB AIRI Team

Motorcycle Traffic Violation Detection System · PITB AIRI Team AI Internship Task 1 · Custom Domain

Option E

Model: YOLOv8n · mAP@0.5: 0.926 · Dataset: 578 images · Platform: Google Colab Tesla T4